

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Computer Graphics

Progress Report

Assigment 1

Advisor(s):	Trần Thị Ngọc Trâm	
Student(s)	Lê Công Minh Khang	2252295
	Philippe Belda	2660010

HO CHI MINH CITY, 29 SEPTEMBER 2026

Table of Contents

I.	Software structure	3
II.	Shape Generators	3
II.1.	Data format	3
II.2.	Obj files	3
II.3.	2D shapes	4
II.3.1.	__circularish helper function	4
II.3.2.	Simple 2D shapes (triangle, rectangle, trapezoid, arrow)	4
II.3.3.	n-gon shapes (pentagon, hexagon, circle, ellipse)	4
II.4.	3D shapes	5
II.4.1.	triangulationNation helper function	5
II.4.2.	Cube	5
II.4.3.	Simple __circularish and triangulationNation shapes (cylinder, cone, truncated cone, tetrahedron, prism)	6
II.4.4.	Sphere	6
II.4.5.	Torus	6
II.4.6.	Surface	7
III.	Shaders	8
IV.	Lighting	8
V.	GUI	9
VI.	Performance	9
VII.	Challenges encountered	9

I. Software structure

The software has been structured to ensure modularity and concern separation. A first module is tasked to get the scene informations and pass it to the window manager. This modules handles the creation of the GL context the window management (display and hud). The draw calls at each frame are managed by the pipeline module that stores scene information as well as the wiring with GPU.

In Figure 1 is provided a more detailed diagram to better understand the sequence of operations and the responsibilities of each module.

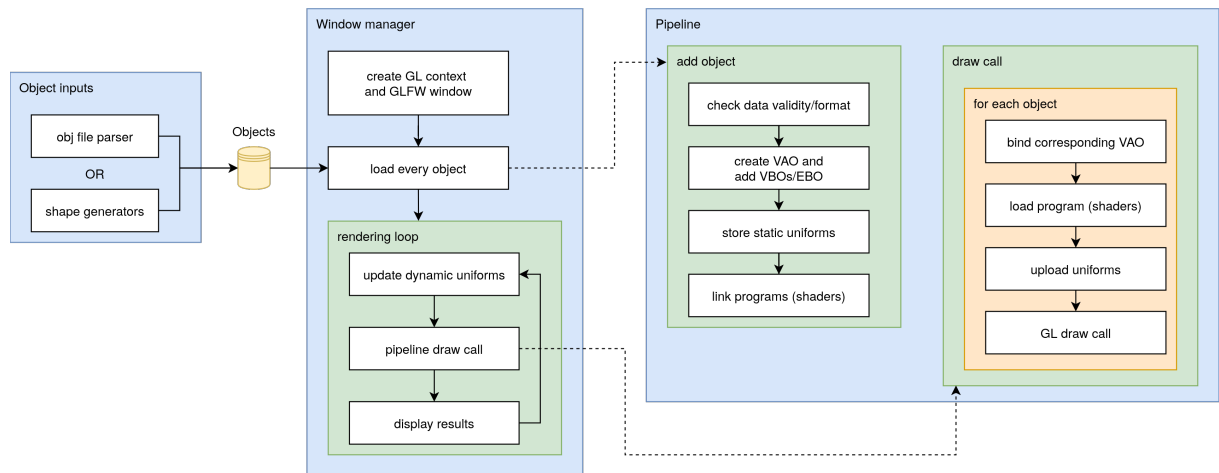


Figure 1: Diagram of the software structure

II. Shape Generators

II.1. Data format

We chose to bundle object data for rendering under the form:

```
data = {
    "vertices" # np.array((n, 3))
    "indices" # (optional): np.array((n)) if omitted, will be picking k-tuples
from the vertices array where k is the size of primitive (eg 3 for triangles)
    "colors" # np.array((n, 3)) each component must be in [0, 1]
    "normals": # np.array((n, 3))
    "mode": # GL mode, indicating the rendering mode (eg: GL_TRIANGLES,
GL_TRIANGLE_STRIP, ...)
```

All shape generators and .obj files parsing outputs are converted to this format. At the moment, for the sake of simplicity, the rendering mode is always GL_TRIANGLES, and colors and normals are automatically populated.

II.2. Obj files

From the wiki, we can see that .obj files are text files with the syntax:

```
<fieldtype> <field1> <field2> <etc..>
v 0.123 0.234 0.345 1.0
...
```

```

vt 0.500 1 [0]
...
vn 0.707 0.000 0.707
...
vp 0.310000 3.210000 2.100000
...
f 1 2 3
f 3/1 4/2 5/3
f 6/4/1 3/5/3 7/6/5
f 7//1 8//2 9//3
f ...

```

For now, we shall only focus on the 2 fields v (Vertex) and f (Triangle):

- For `data["vertices"]`, we parse the “v” fields as is
- For `data["indices"]`, the “f” fields in `.obj` files are 1-indexed when referencing which vertex is part of a triangle. As such, converting “f” fields to “indices” will need to decrement all referenced indices by 1. We collect only the X number in each pattern X/Y/Z.

II.3. 2D shapes

II.3.1. `__circularish` helper function

Generating circles comes up a few times in some of the 2D and 3D shapes generation methods, thus a helper function like `__circularish` is very convenient.

The exact functionality and implementation varies depending on use cases, which is why the function is copied accross files instead of declared and imported. In general, it takes in:

- `n`: number of vertices generated
- `size`: the distance the vertices will be from (0, 0, z)
- `z`: the z coordinate of vertices

It follows:

Given step size in angle:

$$\theta = \frac{2\pi}{n}$$

for `i` in `range(n)` we create a new vertex at coordinate:

$$(\cos(i\theta) \times \text{size}, \sin(i\theta) \times \text{size}, z)$$

II.3.2. Simple 2D shapes (triangle, rectangle, trapezoid, arrow)

These shapes are relatively simple, so each generator only takes as parameters a few basic arguments to scale them in different aspects. The vertices then gets their coordinate multiplied accordingly and populating `data["indices"]` is hardcoded.

II.3.3. n-gon shapes (pentagon, hexagon, circle, ellipse)

All of these shapes can be generated with some variation of the `__circularish` function. Notably:

- Pentagon: $n = 5$
- Hexagon: $n = 6$
- Circle: $n = 60$
- Ellipse: $n = 60$, with a multiplier to the width of the n-gon in one of the axis

These rings of vertices are triangulated to all have 1 common vertex, and then iterate around the n-gon for the other 2 vertices. This method is simple and ensures the correct ordering of the indices but can lead to rendering artifacts (pinching) around the common vertex. A better way would be to do it in zigzag strips, or manually implementing the GL_TRIANGLE_FAN mode (recall that we assume that all shapes will always render in GL_TRIANGLES mode).

II.4. 3D shapes

II.4.1. triangulationNation helper function

A lot of 3d shapes are more conveniently generated as quads (4 sided polygon). However, to be able to render in mode GL_TRIANGLES, we need to convert these lists of quads into a list of triangles.

It follows:

```
#      0.-----3
#      | \    |
#      |  \   |
#      |   \  |
#      |    \ |
#      1'-----'2
# inputQuad = [0,1,2,3]
res = []
for quad in listOfQuads:
    res.append([[quad[0],quad[1],quad[2]]])
    res.append([[quad[0],quad[2],quad[3]]])
return res
```

Note that the method requires that quads part of the input list needs to have vertices indexed in a counter clockwise direction. Usage of this method would need some consideration in how the listOfQuads are generated.

II.4.2. Cube

The simplest 3d shape, establishes the generation pattern that is used further down the line.

```
p = size/2
obj["v"].append([ -p ,  p ,  p])
obj["v"].append([ -p ,  p , -p])
obj["v"].append([ -p , -p ,  p])
obj["v"].append([ -p , -p , -p])
obj["v"].append([  p , -p ,  p])
obj["v"].append([  p , -p , -p])
obj["v"].append([  p ,  p ,  p])
obj["v"].append([  p ,  p , -p])
```

```

listOfQuads = []
# top
listOfQuads.append([0,2,4,6])
# bottom
listOfQuads.append([7,5,3,1])

for i in range(0,8,2):
    listOfQuads.append([(i+0)%8,(i+1)%8,(i+3)%8,(i+2)%8])

```

We initialize vertices in pairs that creates 4 parallel edges of the cube going counter clockwise (in the code shown above it is parallel to the z axis).

Then quads of the top and bottom faces of the cube are hard coded, paying attention to “reverse” the order for the bottom face to ensure that it faces the correct way (backface culling).

Finally, as the order of vertex declaration is suitable, we can use a loop to iterate over the 4 side faces of the cube, with modulo operation % to be able to connect the last vertex pair to the first.

II.4.3. Simple `__circularish` and `triangulationNation` shapes (cylinder, cone, truncated cone, tetrahedron, prism)

All of these shapes are generated in a process similar to the Cube, as some combination of `__circularish` for the top/bottom faces and a loop to generate quads as side walls:

- Cylinder: 2 n-gon top/bottom faces with a list of quads connecting the two
- Prism: 2 triangle top/bottom faces with a list of quads connecting the two
- Truncated cone: a Cylinder with 1 of the circles scaled down
- Cone: an n-gon pyramid, with an n-gon bottom face and a triangle fan as the side connecting to a vertex at the top
- tetrahedron: a specifically scaled cone with a triangle as a base

II.4.4. Sphere

There are multiple ways to create a sphere, we chose uv sphere due to the similarity in construction with the earlier shapes.

In which, a sphere contains multiple `__circularish` n-gons of various sizes and z offsets, akin to the latitudes of a globe. These rings are connected to adjacent loops akin to how the sides of a cylinder was generated, only for multiple levels. At the pole, the nearest n-gon to the pole vertex is connected similar to how the cone was generated.

II.4.5. Torus

A Torus is a shape much like a doughnut, or a ring you wear on your hand, or a disc with a hole in it, or a chain link. But for rendering, most people would have this ring be rounded, not a flat shape like a disc, or a flat shape (in a another orthogonal direction). You would also want this shape to be evenly thick and smooth not like a slightly bumpy bread surface of a doughnut.

With this requirement of what a torus should look like in the end, one must imagine how to create such a shape. For the roundedness, we can state this more precisely by saying, cutting the torus in half (bisecting) through the center of the holed disc (like breaking a doughnut in half to share with a friend) and have its cross section be the roundest shape: a circle. In fact, you'd want the cross section of the torus be a circle regardless of where you break it in half. In conclusion you must think of revolving a circle around some center axis, lets say the z axis in a 3d x y z coordinate system.

Because we are working with computer graphics, it makes more sense to only generate a few slices, a few angles of the torus bisection to create a circular cross section, to avoid having to populate and render an infinite amount of vertices. So each "circular cross section" would be an **n-gon** shape and around the torus, there should only be **m** cross sections.

Because the programmer is bad at trigonometry and is bad at other college level math classes, it is difficult to generate an **n-gon** cross-sections at different angles of bisection, we do it another way. Instead of generating **m n-gon** cross sections, we can generate 1 piece of each **m** cross sections **n** times. To motivate this approach, I want you to imagine cutting the doughnut along its planer plane (for making a sandwich). You will see that the cross section now will be composed of 2 circles centered on the center of the doughnut's hole (one for the outside edge of the doughnut, one for the inner edge). So you could imagine generating these **m** pieces with an `__circularish` generator producing **m-gon** that is centered on the axis of revolution (the z axis). These **m-gons** would have its radius and z offset defined by picking a point along the surface of the doughnut and applying its distance from the z axis and its z offset from the xy plane to the entire **m-gon**.

This means, what we do, is first generate a temporary **n-gon** offset from the z axis to borrow the measurements of its vertices to generate our **m-gons** making up the structure of our torus. The only thing left is some clever loops to connect these **m-gons**' vertices to triangulate the surface of the torus.

II.4.6. Surface

Surface rendering takes in a function $z = f(x,y)$ where $f(x,y)$ is a python lambda: example:

```
func = lambda x, y: numpy.sin(x) + numpy.cos(y)
```

It also takes in parameters to control the limits of the surface to be shown in the x and y direction (`+-limx` and `+-limy`).

Then we iterate `ix, iy` over the ranges of `[-limx,limx]` and `[-limy,limy]` creating vertices with the coordinates `(ix,iy,lambda(x,y))` and connect them with eachother using `triangulationNation`. Note that this implementation will not handle illegal or limits and can't render (or render accurately) discontinuous functions.

III. Shaders

In a given scene, each object possesses its own shader program. A shader program is created from 2 shader: a vertex shader and a fragment shader. Specifically, during object creation (in `Pipeline.add_object`), the 2 shader files are uploaded to the GPU then compiled. Then there are linked into a program, associated with the object. Then the 2 former shader objects are destroyed.

Thus remains a program for each object.

However, the shaders might need some associated data: uniforms and textures. The `UManager` class is charged to handle these.

- uniforms:

The uniforms are separated in 2 categories: static and dynamic.

As the name suggests, the static ones stay the same during the whole execution of the program, while the dynamic ones change from frame to frame.

There are currently only 2 dynamic uniforms: one to track the time passed from the start of the execution (`float iTime`), and another to track the mouse position (`vec4 iMouse`). Dynamic uniforms are computed for each frame and are declared in `windows.py`, thus this type of uniforms is not available for the final user. However, static uniforms can be defined by the user and passed as parameters in `Pipeline.add_object` during the creation of each object.

- textures:

Currently textures are not supported.

IV. Lighting

The lighting of a scene can completely change its final rendering, hence it plays a major role in computer graphics. In this program, we chose to handle scene lights thanks to a data structure that is passed to the fragment shader as a uniform. This uniform is in fact an array of lights, to enable the definition of several of them. A light is declared in python as an instance of the class `Light`, that is simply a container to hold the attributes defining the light.

Currently, the class is as follow:

```
class Light:
    def __init__(
        self,
        position,
        color=(1, 1, 1),
        intensity=1.0,
        # dtype=1,
        # constant=1.0,
        # linear=0.09,
        # quadratic=0.032,
    ):
        self.position = position
        self.color = color
        self.intensity = intensity
        # self.dtype = dtype
```

```
# self.constant = constant
# self.linear = linear
# self.quadratic = quadratic
```

With the light passed down to the fragment shader, we simply have to apply a formula (Goureaud, Phong, ...) to obtain the fragment color.

V. GUI

In this version, the graphical user interface (GUI) is limited to a window displaying the OpenGL rendering. It supports mouse interaction through the use of camera, but no head-up display (HUD) or keyboard interaction.

VI. Performance

Performance and frame rate has not been yet investigated.

VII. Challenges encountered

The integration of cameras inside the project architecture raised some questions about the project modularity and organisation. Indeed, cameras are needed to represent a 3D scene and one could decide to not provide any support from the library and ask the user to compute the view and projection matrices directly inside the vertex shader. This raises concerns about inputs, that then have to pass through uniforms, and about debugging. Indeed, debugging a Python code is much more convenient than debugging one written in GLSL, especially when it comes to matrix computations and hardware compatibility issues.

This reasons led us to create the `Camera` class, a pattern providing 3 main functions: `view_matrix`, `_projection_matrix` and `update`. The first 2 return the corresponding matrices and the latter returns an array of uniforms updated with the “projection” and “view” fields.

Then, once a camera implementation is passed to the main program, the user simply has to collect the “view” and “projection” uniforms in the vertex shader to compute the final vertices position.

List of Figures

Figure 1 Diagram of the software structure	3
--	---